

第 4 章 实验项目

4.1 函数信号发生器的设计、组装与调试研究

一、实验目的

- 1、学习和掌握函数信号发生器的基本原理；
- 2、掌握 Arduino 编程原理；
- 3、学习和掌握 PCB 电路板的设计和制作；
- 4、掌握波形参数的测试方法。

二、实验原理和电路设计

在开始介绍项目之前，先说明一下什么是函数信号发生器。

函数发生器是一种能产生某种波形的电路，常见的有四种波形：方波、三角波、锯齿波和正弦波。

一个专用的函数信号发生器将花费上千元以上，当然要比我们在这里构建的系统功能要强大得多，我们这里只是给大家去进行一些有趣实验尝试，掌握其基本原理。

1、方波

采用 Arduino 产生简单的波形为方波，方波是高低逻辑电平之间简单循环。

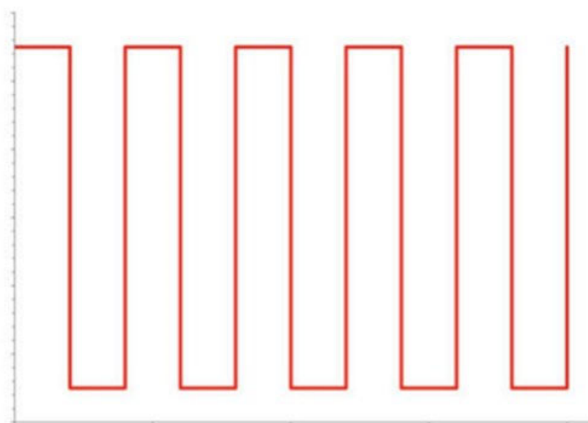


图 4.1 方波示意图

我们用来产生方波的电路几乎是可以想象的最平常的电路。

事实上，我们需要做的只是连接 Arduino 的一个数字输出引脚（这里用 8 脚）和地。代码也很简单：

```
void setup()
{
  pinMode(8,OUTPUT);
}
void loop()
{
  // A total delay of 4 ms (frequency 250Hz)
  digitalWrite(8,HIGH);
  delay(2);
```

```
digitalWrite(8,LOW);
delay(2);
}
```

我们还可以使用 `delaymicroseconds()` 产生较小的延迟值。

事实上，Arduino 具有比这更容易产生一个方波的函数。例如，产生我们之前 250Hz 方波，我们可以使用：`tone(8, 250)` 代替 `digitalWrite` 语句。

我们可从在示波器上，看到这个方波到底是什么样子：



图 4.2 方波波形（占空比 50%）

除了波的频率，还可以看看波的对称性，即所谓的占空比。

占空比只是测量高电平时间占总周期时间的比例。若方波是对称的：高低电平两种状态下都有相同的时间，即占空比为 50%。

如果我们要改变占空比，我们不能用 `tone()` 功能的设计。我们需要使用 `digitalwrite()` 和 `delay()` 功能。例如，产生一个占空比为 25% 的 250Hz 方波，在高电平后延迟 1 毫秒，然后低电平后延迟后 3ms。

使用示波器，我们就能看到波形的形状。



图 4.3 方波波形（占空比 25%）

时间坐标每格 2ms，我们可以清楚地看到脉冲宽度 1ms，脉冲之间的间隙为 3ms。如万用表测量电压，一个 50%占空比的 250Hz 信号。

2、三角波

三角波是一种具有特定波形的周期信号。它的波形类似于一个等腰三角形，因此得名。在三角波中，信号值会沿着一个方向线性地增加，直到达到峰值，然后再沿相反的方向线性地减小，形成一个周期。三角波在音频合成、信号调制、电子测试以及各种电子设备中都有广泛的应用。在实际应用中，三角波可以通过集成电路或数字信号处理器等方式产生，还可以使用数字模拟转换器（DAC）来产生。

DAC 是一种将数字信号转换为模拟信号电子器件，它的输出可以表示为一个模拟的连续信号，这个信号的形状可以通过输入的数字代码控制。因此，我们可以通过向 DAC 输入特定的数字代码，来让它输出对应形状的模拟信号，例如三角波、正弦波、方波等。在产生三角波时，DAC 的输出电压会以一定的速率线性地增加或减小，从而形成三角波的波形。当然，产生高精度的三角波需要高分辨率的 DAC，并且需要精确的时钟同步和滤波电路来消除噪声和抖动。下面介绍一种简单 DAC。

R-2R 电阻梯形网络，是一种常见的数字到模拟信号的转换方法。R-2R 电阻梯形网络利用一系列电阻来实现不同权重的电压输出，以将数字输入值转换为模拟电压输出。R-2R 电阻梯形网络是一种经济有效的方式，用于将数字信号转换为模拟信号，通常用于低到中等分辨率的应用，如音频和控制系统。较高分辨率的 DAC 可能需要其他技术来实现更高的精度和性能。

以下是 R-2R 电阻梯形网络的原理和工作方式：

电阻网络：R-2R 电阻梯形网络通常由两种电阻组成：R 和 2R。R 表示一种电阻值，而 2R 表示另一种电阻值，通常是 R 的两倍。这些电阻连接成一个网络，用于将二进制数字输入值（0 或 1）转换为相应的电压。

二进制输入：DAC 的输入是一个 n 位的二进制数字（0 和 1 组成的数字）。每位二进制数字控制一个开关，以确定是否将相应的电阻连接到地（0）或参考电压（1）。

电压输出：通过电阻梯形网络，将所有开启（1）的电阻连接到参考电压，而将所有关

闭（0）的电阻连接到地。然后，根据电阻分压原理，通过这些电阻的组合，可以计算出一个合成的模拟电压输出。

对于一个 n 位的 R-2R 电阻梯形网络，输出的模拟电压可以通过以下公式表示：

$$V_{out} = (D_{n-1} \times 2^{n-1} + D_{n-2} \times 2^{n-2} + \dots + D_1 \times 2^1 + D_0) \times (V_{Ref}/2^n) \quad (4-1)$$

其中， V_{out} 是输出的模拟电压， D_{n-1} 到 D_0 是输入的二进制代码（每一位为 0 或 1）， V_{Ref} 是参考电压。

这个公式表示了将二进制代码转换为模拟电压的过程。每一位二进制代码通过乘以对应的权重（2 的幂次方）后进行累加，然后再除以总位数的 2 的幂次方，最终乘以参考电压 V_{Ref} ，得到输出的模拟电压。

权重和分辨率：在 R-2R 网络中，每位的电阻（ $2R$ 或 R ）具有不同的权重，这意味着每位的电压输出对最终的模拟输出有不同的贡献。最低有效位（LSB）的权重最小，而最高有效位（MSB）的权重最大。参考电压 V_{Ref} 和位数 n 决定了 DAC 的分辨率，即能够表示的不同模拟电压级别的数量。

运算放大器：在实际应用中，R-2R 网络的输出通常由一个运算放大器（Op-Amp）进行缓冲和放大，以提供所需的输出电流或电压。

精度和线性度：R-2R 电阻梯形网络的精度和线性度受到电阻值匹配的影响。为了获得更高的精度，电阻值应该非常匹配，以确保权重准确。

用 DAC 如何产生一个三角形波形，如下图所示。

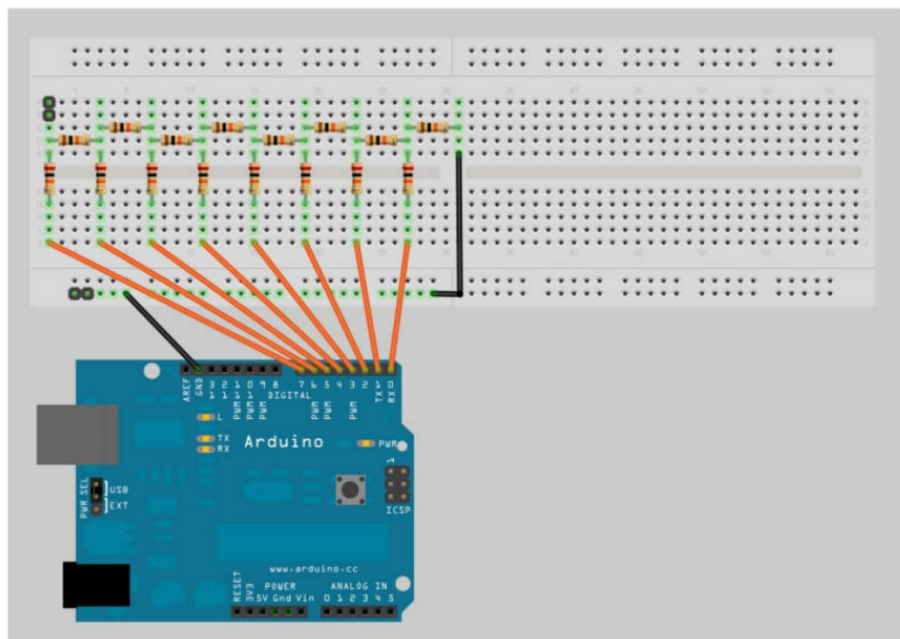


图 4.4 R-2R 电阻梯形网络

这是一个 8 位 DAC 电路称为 R-2R 电阻梯形网络。八位中的每一位对输出电压都有贡献。输出电压可根据前面的公式（4-1）计算。

产生三角波形程序为：

```
void setup()
{
  pinMode (0, OUTPUT);
  pinMode (1, OUTPUT);
  pinMode (2, OUTPUT);
  pinMode (3, OUTPUT);
```

```

pinMode (4, OUTPUT);
pinMode (5, OUTPUT);
pinMode (6, OUTPUT);
pinMode (7, OUTPUT);
}
void loop()
{
for (int i=0; i<255; i++)
{
    PORTD=i ;
}
for(int i=255;i>0;i--)
{
    PORTD=i ;
}
}

```



图 4.5 三角波波形

注意，这里我们使用 **PORTD** 代替单个输出引脚同时设置状态。这样更快，并且确保所有的引脚在同一时间改变，这对产生平滑的模拟电压十分重要。

PORTD 是寄存器 **D** 端口，在这里其作用，是因为它映射到 0-7 数字引脚（8 位）；在一个操作，我们控制所有 8 引脚将二进制或十进制值的写入寄存器。例如，如果我们赋十进制值 123（这相当于 B01111011），将置 6，5，4，3，1，0 为高电平。

虽然引脚 0、1 是用于串行收发，但使用 **PORTD** 可以同时操作 8 个输出引脚不影响串行通信。

另外，我们可以非常容易地修改我们的代码，通过简单地注释（或者删除）**loop** 中的第二个 **for** 语句来产生锯齿波形。

3、正弦波

对于正弦波。我们需要预先计算随着时间推移的输出电压值（Arduino 不足以快到实时获取它们）。

正弦波一个周期为 2π 弧度——所以为了产生一个每个周期 256 时间步长的的正弦波，对每个 x 值，计算 $y = \sin((x / 255) * 2\pi)$ 。因为正弦函数在 1 和 -1 间取值，而我们想要 0 和 255 之间的值，最简单的方法就是将浮点值乘以 128——给出一个 -128 到 128 间的值，然后加上 128 从而数值落在正确的范围内。

在 `setup()` 阶段，在全局数组存储结果。那么在 `loop` 中，我们需要做的是通过一次数组循环。程序代码如下：

```
#define PI 3.14
int sine[255];
void setup()
{
  pinMode(0,OUTPUT);
  pinMode(1,OUTPUT);
  pinMode(2,OUTPUT);
  pinMode(3,OUTPUT);
  pinMode(4,OUTPUT);
  pinMode(5,OUTPUT);
  pinMode(6,OUTPUT);
  pinMode(7,OUTPUT);
  float x;
  float y;
  for(int i=0;i<255;i++)
  {
    x = (float) i;
    y =sin((x/255)*2*PI);
    sine[i]=int (y*128)+128;
  }
}
void loop()
{
  for(int i=0;i<255;i++)
  {
    PORTD=sine[ i ];
    delayMicroseconds(10);
  }
}
```

如果我们运行这个代码，并用 DAC(如 R-2R 电阻梯形网络)，在示波器上我们就会发现波形确实非常像正弦波。

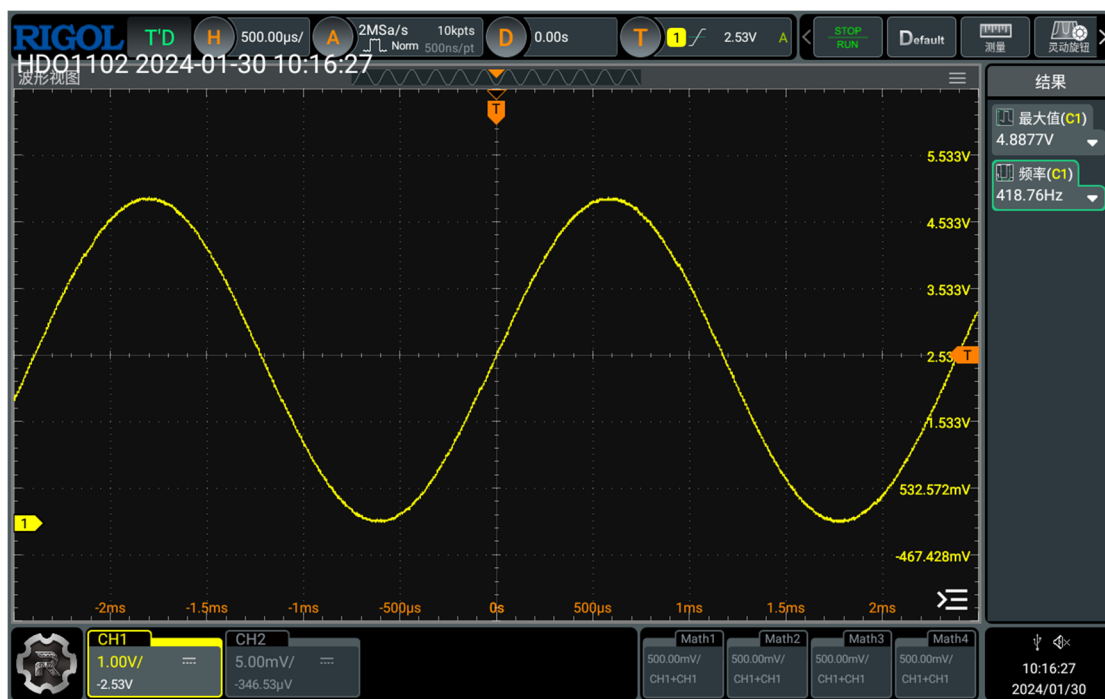


图 4.6 正弦波波形

三、实验任务与要求

- 1、用 Arduino UNO 设计一个函数信号发生器，要求如下：
 - (1) 输出可在方波、三角波、正弦波间切换。
 - (2) 输出频率自定。
 - (3) 输出幅度 0.5~2V 可调（要求加输出缓冲）。
- 2、设计的电路，调整元件参数，直至满足任务要求。重点考虑：
 - (1) 如何切换波形（硬件、软件方式）；
 - (2) DAC 方法（不限于 R-2R 电阻梯形网络）；
 - (3) 单电源运放缓冲设计。
- 3、绘制程序流程图，编制与调试函数信号发生器程序。
- 4、制作 Arduino 扩展板，并将其与 UNO 板组装联调。

四、思考题

- 1、如何改变方波的占空比？
- 2、试分析简单的函数信号发生器中影响波形频率的因素；
- 3、简述 R-2R 电阻梯形网络实现 DAC 的原理。